# Comprehensive Technical Report

## Energy Knowledge Graph Pipeline, Semantic Reasoning, KGE, and RAG Chatbot

Web Mining & Semantics Project
OUALI Youssef
SMASH Yassine

March 23, 2026

**Abstract**

This report documents, in implementation-level detail, the full project developed for the Web Data Mining and Semantics course. The system follows a complete semantic pipeline: (1) web crawling and cleaning in the sustainable energy domain, (2) named entity and relation extraction, (3) RDF knowledge graph construction and Wikidata alignment, (4) external knowledge base expansion through SPARQL neighborhood traversal, (5) rule-based reasoning using SWRL-style logic and ontology tooling, (6) knowledge graph embedding experiments (TransE and RotatE), and (7) a retrieval-augmented generation (RAG) question-answering interface based on local LLM-driven SPARQL generation.

The implementation is production-oriented for a course project: modularized source code under `src/`, script orchestration in `run_pipeline.py`, generated artifacts under `data/` and `kg_artifacts/`, and both CLI and Web UI demos for final interaction. This report describes architecture, algorithms, important hyperparameters, design decisions, outputs, quantitative results, observed limitations, and reproducibility steps.

## Contents

**1 Project Context and Objectives** **3**
1.1 Course Scope . . . 3
1.2 Main Goal . . . 3
1.3 Technical Deliverables . . . 3

**2 Repository Organization and Execution Flow** **3**
2.1 Directory Layout . . . 3
2.2 Pipeline Orchestration . . . 4

**3 Environment and Dependencies** **4**
3.1 Runtime . . . 4
3.2 Major Libraries . . . 4

**4 TD1: Web Mining and Information Extraction** **5**
4.1 Focused Crawler Design (`src/crawl/crawler.py`) . . . 5

4.2 Crawling Results . . . 5
4.3 NER and Relation Extraction (`src/ie/ner_extractor.py`) . . . 5

**5 TD4: Knowledge Graph Construction, Alignment, and Expansion** **6**
5.1 Private KG Construction (`src/kg/build_kg.py`) . . . 6
5.2 Entity Alignment to Wikidata . . . 6
5.3 Predicate Alignment . . . 7
5.4 KB Expansion (`src/kg/expand_kb.py`) . . . 7

**6 TD5: Symbolic Reasoning and Knowledge Graph Embeddings** **8**
6.1 Part A — SWRL on Family Ontology . . . 8
6.2 Part B — Rule-Based Reasoning on the Energy KG . . . 8
6.3 KGE Data Preparation (`src/kge/prepare_kge.py`) . . . 8
6.4 KGE Training and Evaluation (`src/kge/train_kge.py`) . . . 8
6.5 Size Sensitivity and Embedding Diagnostics . . . 9

**7 TD6: RAG over RDF Graph with Local LLM** **9**
7.1 System Overview . . . 9
7.2 Pipeline Steps per Question . . . 9
7.3 No-RAG Baseline vs KG-RAG . . . 9
7.4 User Interfaces . . . 10

**8 Artifact Inventory (What Each Output Means)** **10**

**9 Engineering Decisions and Rationale** **11**
9.1 Why This Architecture . . . 11
9.2 Quality and Robustness Mechanisms . . . 11

**10 Observed Limitations** **11**

**11 Potential Improvements (Future Work)** **12**

**12 Reproducibility Guide** **12**
12.1 Install . . . 12
12.2 Run Complete Pipeline . . . 12
12.3 Run Step-by-Step . . . 12
12.4 RAG Web Demo . . . 12

**13 Conclusion** **12**

# 1 Project Context and Objectives

## 1.1 Course Scope

The project integrates multiple course themes in one consistent workflow:

- Web mining and focused crawling,
- Information extraction from unstructured text,
- Semantic modeling in RDF/OWL,
- Ontology alignment and knowledge base enrichment,
- Symbolic reasoning (SWRL-style rule logic),
- Representation learning over graphs (KGE),
- Retrieval-augmented QA over a local KG.

## 1.2 Main Goal

Build an end-to-end **energy-domain semantic system** that starts from web documents and ends in a question-answering assistant able to retrieve factual answers from a structured KG.

## 1.3 Technical Deliverables

The pipeline is implemented as reusable Python modules and runnable scripts. Core outputs include:

- crawled content and crawl metadata,
- extracted entities and relation candidates,
- aligned/expanded RDF graphs,
- reasoned ontology output,
- train/valid/test files for embedding,
- KGE metrics and analysis artifacts,
- RAG CLI and Flask web application.

# 2 Repository Organization and Execution Flow

## 2.1 Directory Layout

The project is organized into:

- `src/crawl`: crawler and document acquisition,
- `src/ie`: NER + dependency-based relation extraction,
- `src/kg`: KG construction, alignment, and expansion,

- `src/reason`: SWRL/fact inference logic,
- `src/kge`: PyKEEN preparation and training,
- `src/rag`: SPARQL-generation RAG and web UI,
- `data`: intermediate and evaluation tables,
- `kg_artifacts`: RDF/OWL artifacts.

### 2.2 Pipeline Orchestration

The script `run_pipeline.py` defines deterministic stages:

1. **crawl** → `crawler_output.jsonl`
2. **ie** → `extracted_knowledge.csv`, `relation_candidates.csv`
3. **kg** → `alignment.ttl`, mapping CSVs
4. **expand** → `expanded.ttl`, `expanded.nt`
5. **reason** → `ontology.ttl`
6. **kge** → split files + `kge_results.csv`
7. **rag** → interactive local QA (CLI / web)

This allows either full execution (`-step all`) or isolated stage execution for debugging and incremental development.

## 3 Environment and Dependencies

### 3.1 Runtime

- Python 3.10+,
- local Ollama server for LLM inference,
- CPU-compatible operation for all steps (GPU optional for speed).

### 3.2 Major Libraries

The implementation uses:

- **Crawling/cleaning:** `trafilatura`, `httpx`,
- **IE/NLP:** `spaCy` (prefers `en_core_web_trf`, falls back to smaller models),
- **Semantic stack:** `rdflib`, `SPARQLWrapper`, `owlready2`, `owlrl`,
- **Embedding and ML:** `pykeen`, `torch`, `scikit-learn`, `matplotlib`,
- **App layer:** `Flask`, `requests`.

# 4 TD1: Web Mining and Information Extraction

## 4.1 Focused Crawler Design (`src/crawl/crawler.py`)

**Seed Strategy:** 12 Wikipedia pages are used as high-signal seed documents for renewable/-sustainable energy and energy governance topics.

**Robots Policy:**

- robots.txt checked per host via `urllib.robotparser`,
- custom user agent (academic context),
- graceful behavior if robots file cannot be fetched (default permissive fallback).

**Content Extraction:**

- HTML fetched via Trafilatura,
- boilerplate removed with precision-oriented extraction,
- whitespace normalization,
- quality gate: `MIN_WORDS = 500`.

**Outputs:**

- `data/crawler_output.jsonl`: high-quality pages,
- `data/crawl_summary.csv`: all URL statuses and counts.

## 4.2 Crawling Results

From `crawl_summary.csv`:

- Total seed URLs: **12**
- Successful pages (status `ok`): **12**
- Aggregate extracted words: **110,340**

## 4.3 NER and Relation Extraction (`src/ie/ner_extractor.py`)

**NER labels retained:** `PERSON`, `ORG`, `GPE`, `DATE`.

**Entity filtering:**

- minimum text length,
- acronym/noise suppression,
- stop-list for units and frequent non-entity patterns,
- per-source deduplication.

**Relation extraction heuristic:**

- sentence-level dependency parsing,
- select verbal `ROOT`,
- subjects from `nsubj`/`nsubjpass`,
- objects from `dobj`/`attr`/`pobj`/`obj`,
- map tokens back to detected entities,
- generate triples $(subject, predicate_lemma, object)$.

**IE outputs and scale:**

- `extracted_knowledge.csv`: **6,966** entity rows,
- `relation_candidates.csv`: **37** deduplicated relation rows.

**Most frequent extracted predicates (top examples):** `produce (3), retrieve (3), become (3), declare (2), approve (2), set (2)`.

# 5 TD4: Knowledge Graph Construction, Alignment, and Expansion

## 5.1 Private KG Construction (`src/kg/build_kg.py`)

**Namespace scheme:**

- Entities: `http://private.org/energy/entity/...`
- Relation properties: `http://private.org/energy/prop/...`
- Types: `http://private.org/energy/type/...`

**Construction process:**

1. Read entities and relation candidates from TD1 outputs,
2. slugify labels to deterministic URI-safe identifiers,
3. assert labels and type triples,
4. materialize extracted relation triples as RDF edges.

## 5.2 Entity Alignment to Wikidata

**Method:**

- collect and frequency-rank candidate labels,
- query Wikidata `wbsearchentities` API,
- select top candidate QID,
- attach `owl:sameAs` links,

- persist mapping table with confidence scores.

**Configuration and safeguards:**

- cap alignment labels: `max_labels = 500`,
- retry logic for API failures,
- confidence heuristic based on lexical overlap.

**Observed alignment statistics:**

- rows in `entity_mapping.csv`: **500**
- non-empty Wikidata links: **340**
- linking coverage over attempted labels: **68.0%**
- average confidence on linked rows: **0.9231**

### 5.3 Predicate Alignment

A manual mapping dictionary links frequent verbal relations to Wikidata direct properties (e.g., `produce` → `wdt:P1056`, `use` → `wdt:P2283`, `join` → `wdt:P463`, `publish` → `wdt:P123`).

**Output:** `data/predicate_alignment.csv` with **4** aligned predicates in this run.

### 5.4 KB Expansion (`src/kg/expand_kb.py`)

**Goal:** densify the private graph via one-hop Wikidata expansion.

**Algorithmic choices:**

- frontier-based BFS over aligned entity seeds,
- confidence filter: `MIN_CONF = 0.75`,
- one-hop query restricted to `wdt:` predicates and entity objects,
- per-entity edge cap: `PER_ENTITY_LIMIT = 50`,
- budget constraints: `TARGET_TRIPLES = 55,000`, `MAX_REQUESTS = 4,000`.

**Produced artifacts:**

- `kg_artifacts/expanded.ttl`
- `kg_artifacts/expanded.nt`
- `data/kb_stats.csv`

**Expansion result:**

- total triples: **55,007**
- total entities: **28,285**
- total relation URIs: **477**

# 6 TD5: Symbolic Reasoning and Knowledge Graph Embeddings

## 6.1 Part A — SWRL on Family Ontology

In `src/reason/swrl_reasoning.py`, the project loads `kg_artifacts/family_lab.owl` and applies a SWRL rule equivalent to:

$$\texttt{Person(?p)} \wedge \texttt{age(?p,?a)} \wedge \texttt{greaterThan(?a,60)} \rightarrow \texttt{oldPerson(?p)}$$

Reasoning is executed with HermiT through `owlready2`, and inferred individuals are printed.

## 6.2 Part B — Rule-Based Reasoning on the Energy KG

The same script applies a Horn-rule-style inference on the project graph using SPARQL `CONSTRUCT`:

$$\texttt{produce(?X,?Z)} \wedge \texttt{locate(?X,?Y)} \rightarrow \texttt{hasEnergyProducer(?Y,?X)}$$

The enriched graph is saved as `kg_artifacts/ontology.ttl`. This stage demonstrates symbolic rule design on domain data and creates a bridge to vector-level relation analysis in KGE.

## 6.3 KGE Data Preparation (`src/kge/prepare_kge.py`)

The RDF graph is converted to URI-only triples and split with deterministic shuffling:

- train ratio: **80%**
- validation ratio: **10%**
- test ratio: **10%**

**Generated split sizes:**

- `train.txt`: **39,486**
- `valid.txt`: **4,935**
- `test.txt`: **4,937**

## 6.4 KGE Training and Evaluation (`src/kge/train_kge.py`)

**Models trained:**

- TransE
- RotatE

**Main hyperparameters:**

- embedding dimension: `100`
- epochs: `100` (base experiments)

- batch size: `256`
- learning rate: `0.01`
- early stopper with patience/frequency controls

**Link prediction metrics (filtered):**

| Model | MRR | Hits@1 | Hits@3 | Hits@10 |
|---|---|---|---|---|
| TransE | 0.0475 | 0.0103 | 0.0459 | 0.1107 |
| RotatE | 0.2605 | 0.1592 | 0.3080 | 0.4640 |

Table 1: Full-dataset KGE results from `data/kge_results.csv`.

**Observation:** RotatE clearly outperforms TransE on this graph, especially on ranking quality and top-$k$ retrieval.

### 6.5 Size Sensitivity and Embedding Diagnostics

Additional experiments train TransE on reduced subsets (20k / 50k / full) and export:

- `data/kge_neighbors.txt`: nearest-neighbor semantic inspection,
- `data/kge_tsne.png`: 2D projection of embedding space,
- `data/kge_results.csv`: all experiment rows.

## 7 TD6: RAG over RDF Graph with Local LLM

### 7.1 System Overview

The RAG implementation is in `src/rag/lab_rag_sparql_gen.py` and `src/rag/web_ui.py`. It combines local LLM generation with deterministic KG querying.

### 7.2 Pipeline Steps per Question

1. Build compact schema summary (prefixes, sampled predicates/classes),
2. Prompt Gemma 2B via Ollama to generate SPARQL,
3. Parse and sanitize returned query,
4. Execute against local `rdflib` graph,
5. If execution fails, trigger automatic repair prompt,
6. Synthesize final natural-language answer from result rows.

### 7.3 No-RAG Baseline vs KG-RAG

For each user question, the system provides:

- **Baseline answer:** direct LLM output without graph grounding,
- **RAG answer:** SPARQL-grounded factual output from KG.

This dual output demonstrates the practical value of graph grounding and explicit semantic querying.

### 7.4 User Interfaces

**CLI mode:**

```
python -m src.rag.lab_rag_sparql_gen
```

**Web mode (Flask):**

```
python -X utf8 src/rag/web_ui.py
# then open http://127.0.0.1:5001
```

The web UI presents: baseline answer, final KG answer, generated SPARQL logic (without prefixes for readability), and tabular raw result rows.

## 8 Artifact Inventory (What Each Output Means)

| Artifact | Purpose / Content |
|---|---|
| `data/crawler_output.jsonl` | Cleaned text documents accepted by quality threshold, one JSON object per URL. |
| `data/crawl_summary.csv` | URL-level crawl diagnostics (status, title, word count). |
| `data/extracted_knowledge.csv` | Entity table from NER with semantic type and source URL. |
| `data/relation_candidates.csv` | Candidate S-P-O relation rows from dependency parsing. |
| `data/entity_mapping.csv` | Private label to Wikidata URI alignment with confidence score. |
| `data/predicate_alignment.csv` | Private relation verbs aligned to Wikidata direct properties. |
| `kg_artifacts/alignment.ttl` | Private KG enriched with `owl:sameAs` and `owl:equivalentProperty`. |
| `kg_artifacts/expanded.ttl` | Expanded KG (Turtle serialization) after one-hop Wikidata traversal. |
| `kg_artifacts/expanded.nt` | Expanded KG in N-Triples format for scalable parsing. |
| `kg_artifacts/ontology.ttl` | Reasoning-enriched ontology output (SWRL-style inference stage). |
| `data/kb_stats.csv` | Global graph statistics (triples, entities, relations). |
| `data/train.txt` | KGE train split in TSV triple format. |
| `data/valid.txt` | KGE validation split in TSV triple format. |

| Artifact | Purpose / Content |
|---|---|
| `data/test.txt` | KGE test split in TSV triple format. |
| `data/kge_results.csv` | KGE metrics for all trained configurations. |
| `data/kge_neighbors.txt` | Nearest-neighbor embedding analysis samples. |
| `data/kge_tsne.png` | t-SNE projection of embedding vectors by URI-family color coding. |

# 9 Engineering Decisions and Rationale

## 9.1 Why This Architecture

- Modular code allows replacing any stage independently (e.g., relation extractor or KGE model).
- Explicit data products per stage simplify debugging and grading traceability.
- The same KG supports symbolic reasoning, embedding learning, and retrieval QA.

## 9.2 Quality and Robustness Mechanisms

- retries and throttling for external APIs (Wikidata/Ollama),
- strict word-count gate on crawled content,
- relation and entity deduplication,
- confidence thresholding for alignment seeds,
- fallback model loading in NLP stack,
- SPARQL repair loop when generated queries fail.

# 10 Observed Limitations

- Dependency-based relation extraction remains sparse compared with entity extraction (37 relation rows vs 6966 entities), indicating conservative extraction behavior and sentence structure sensitivity.
- Some mapped entities are noisy or ambiguous because top-1 API search can over-link short-/common labels.
- KGE neighbor examples show semantically mixed clusters in parts of the space, likely due to noisy source relations and heterogeneous node types.
- LLM-generated SPARQL still depends on prompt quality and schema summarization constraints.

# 11 Potential Improvements (Future Work)

1. Replace heuristic relation extraction with supervised/open IE models and relation normalization,
2. Add entity linking disambiguation with context-aware reranking,
3. Introduce ontology constraints (domain/range) before embedding,
4. Expand prompt grounding with richer schema snippets and few-shot SPARQL templates,
5. Add benchmark suite with exact-match or SPARQL-execution correctness per question,
6. Compare additional KGE architectures (ComplEx, DistMult, ConvE) and tune hyperparameters.

# 12 Reproducibility Guide

## 12.1 Install

```
pip install -r requirements.txt
```

## 12.2 Run Complete Pipeline

```
python run_pipeline.py --step all
```

## 12.3 Run Step-by-Step

```
python run_pipeline.py --step crawl
python run_pipeline.py --step ie
python run_pipeline.py --step kg
python run_pipeline.py --step expand
python run_pipeline.py --step reason
python run_pipeline.py --step kge
python run_pipeline.py --step rag
```

## 12.4 RAG Web Demo

```
ollama run gemma:2b
python -X utf8 src/rag/web_ui.py
# open http://127.0.0.1:5001
```

# 13 Conclusion

The project successfully implements an end-to-end semantic AI workflow from web mining to graph-grounded question answering. It demonstrates integration between symbolic AI (RDF/OWL/SWRL-style reasoning), statistical representation learning (KGE), and neural generation (LLM-assisted

SPARQL generation). The final system is not only a prototype for course requirements, but also a reusable foundation for richer domain knowledge assistants.

**Final verified quantitative snapshot of this run:**

- Crawled pages: 12/12 accepted, 110,340 extracted words,
- Entities extracted: 6,966,
- Relation candidates extracted: 37,
- Entity links attempted: 500, linked to Wikidata: 340,
- Expanded KG size: 55,007 triples,
- KGE best model: RotatE (MRR 0.2605, Hits@10 0.4640).